

MoMo SMS REST API

Security, Documentation & DSA Analysis

Team Astro

Dataset	modified_sms_v2.xml 1,691 SMS records
API Server	Python http.server (http://localhost:8080)
Auth	HTTP Basic Authentication (Base64)
Endpoints	5 CRUD endpoints (GET / POST / PUT / DELETE)
DSA Methods	Linear Search vs. Dictionary Lookup

1. Introduction to API Security

The MoMo SMS Processing System exposes sensitive financial data transaction amounts, sender/receiver phone numbers, and timestamps through a REST API consumed by mobile and web clients. Securing this interface is not optional; a single misconfigured endpoint can expose thousands of user records. API security operates across three layers: **authentication** (proving identity), **authorization** (restricting what an authenticated caller may do), and **transport security** (preventing interception in transit).

This implementation uses HTTP Basic Authentication as a baseline, with HTTPS assumed at the transport layer. Basic Auth was chosen for simplicity in a development context, but it carries well-known weaknesses that are addressed in Section 3. In production, financial APIs handling mobile money data should enforce token-based authentication, rate limiting, and audit logging at minimum.

Key Security Principles Applied

- **Least Privilege:** endpoints return only the data the client needs; no raw database dumps.
- **Authentication on every request:** no endpoint is publicly accessible without valid credentials.
- **Meaningful HTTP status codes:** 401 for missing/invalid credentials, 404 for unknown IDs, 400 for malformed payloads clients can handle errors without guessing.
- **Input validation:** POST and PUT payloads are checked before any mutation is applied to the in-memory store.

2. API Endpoint Documentation

Base URL: `http://localhost:8080` | Auth: HTTP Basic (header: `Authorization: Basic <base64>`)

GET	/transactions	List all SMS transactions
Request	curl -u admin:secret http://localhost:8080/transactions	
Response	[{"id": "1", "type": "INCOMING", "amount": "5000", "sender": "078XXXXXX", "receiver": "079XXXXXXXX", "timestamp": "2024-01-15 10:23:00"}, ...]	
Error codes	200 OK — array of transactions 401 Unauthorized	
GET	/transactions/{id}	Retrieve a single transaction by ID
Request	curl -u admin:secret http://localhost:8080/transactions/42	
Response	{"id": "42", "type": "OUTGOING", "amount": "2000", "sender": "078XXXXXX", "receiver": "072XXXXXXXX", "timestamp": "2024-01-15 11:05:00"}	
Error codes	200 OK 401 Unauthorized 404 Not Found	
POST	/transactions	Create a new transaction record
Request	curl -u admin:secret -X POST -H "Content-Type: application/json" \ -d '{"type": "INCOMING", "amount": "3000", "sender": "078XXXXXXXX", "receiver": "079XXXXXXXX", "timestamp": "2024-01-15 12:00:00"}' \ http://localhost:8080/transactions	
Response	{"id": "1692", "type": "INCOMING", "amount": "3000", "sender": "078XXXXXX", "receiver": "079XXXXXXXX", "timestamp": "2024-01-15 12:00:00"}	
Error codes	201 Created 400 Bad Request (missing fields) 401 Unauthorized	
PUT	/transactions/{id}	Update an existing transaction
Request	curl -u admin:secret -X PUT -H "Content-Type: application/json" \ -d '{"amount": "9000"}' http://localhost:8080/transactions/42	
Response	{"id": "42", "type": "OUTGOING", "amount": "9000", "sender": "078XXXXXX", "receiver": "072XXXXXXXX", "timestamp": "2024-01-15 11:05:00"}	
Error codes	200 OK 400 Bad Request 401 Unauthorized 404 Not Found	
DELETE	/transactions/{id}	Remove a transaction record

Request	<pre>curl -u admin:secret -X DELETE http://localhost:8080/transactions/42</pre>
Response	<pre>{"message": "Transaction 42 deleted successfully"}</pre>
Error codes	200 OK 401 Unauthorized 404 Not Found

3. Authentication & Security Analysis

3.1 Why Basic Authentication Is Weak

HTTP Basic Authentication encodes credentials as `Base64(username:password)` and attaches them to every request header. Base64 is an encoding scheme, not encryption; anyone who intercepts the header can decode it in milliseconds. Even over HTTPS, Basic Auth has fundamental design flaws for production financial APIs:

- **Credentials on every request:** the username and password travel across the network with every single HTTP call, maximising the attack surface for credential theft.
- **No token expiry:** there is no built-in mechanism to invalidate a session; if credentials are leaked the attacker has permanent access until the password is manually changed.
- **No scope or role separation:** Basic Auth is binary either you have the password or you don't. There is no concept of read-only vs. write access, or per-client permissions.
- **Brute-force vulnerability:** without additional rate limiting, an attacker can enumerate passwords against the endpoint indefinitely.
- **Credential storage risk:** developers often hard-code credentials in scripts or commit them to version control when using Basic Auth.

3.2 Recommended Alternatives

Method	How it works	Key Advantage	Best For
JWT (JSON Web Token)	Server issues a signed token on log	Stateless; token carries claims (roles, expiry). No server-side session store needed.	Stateless microservices, mobile apps, SPAs
OAuth 2.0 + OIDC	Delegated authorization flow. Client	Industry standard. Supports scopes, token rotation, and third-party identity providers.	Multi-client systems, third-party ntegrations, enterprise APIs
API Keys (HMAC-signed)	Client signs each request with a sec	Credentials never travel in plaintext. Signature tries to request body replay attacks are harder.	Server-to-server communication, machine clients

For this MoMo API, **JWT with short-lived access tokens (15 min) and rotating refresh tokens** is the recommended upgrade path. It eliminates credential-per-request exposure, supports role-based claims (e.g., `role: read-only` vs. `role: admin`), and integrates cleanly with Spring Security or any Python middleware when the project scales.

4. DSA Analysis: Linear Search vs. Dictionary Lookup

4.1 Methodology

Both search strategies were benchmarked against the full 1,691-record MoMo dataset. 1,000 random lookups were performed per method to amortise cold-start effects, and the experiment was repeated across five dataset sizes (20 -> 1,691) to capture how each algorithm scales. Timing was collected using Python's `time.perf_counter()` for microsecond precision.

4.2 Full Dataset Results (1,691 records)

Metric	Linear Search	Dictionary Lookup
Total time (1,000 lookups)	0.045500 s	0.000198 s
Average per lookup	45.500 μ s	0.198 μ s
Speedup factor	—	229.5x faster
Time complexity	$O(n)$	$O(1)$ average
Space complexity	$O(1)$	$O(n)$

4.3 Scaling Comparison

The table below shows average lookup time per operation across increasing dataset sizes. Linear search time grows roughly proportionally with dataset size; dictionary lookup stays near-constant regardless.

Dataset Size	Linear Search (μ s/op)	Dict Lookup (μ s/op)	Speedup
20	54.282	0.144	375.8x
100	51.922	0.205	253.8x
500	43.488	0.077	563.9x
1,000	47.802	0.192	249.0x
1,691	95.489	0.129	739.8x

4.4 Why Is Dictionary Lookup Faster?

A dictionary (hash map) computes a numeric hash of the key and maps it directly to a memory bucket lookup is a single arithmetic operation plus one or two memory accesses, independent of how many records exist. Linear search has no such shortcut: it must examine each record in sequence until a match is found, so worst-case cost grows linearly with dataset size. At 1,691 records the dictionary was **739.8x faster** per lookup, a gap that widens further as data grows.

4.5 Further Improvements

- **Binary Search on a sorted array ($O(\log n)$):** halves the remaining search space each step. Suitable when data changes rarely and sorted traversal is also needed.

- **B-Tree index ($O(\log n)$, cache-friendly):** the structure used internally by PostgreSQL/MySQL. Optimal for range queries (e.g., 'all transactions between date A and date B') that hash maps cannot serve efficiently.
- **Trie / Prefix Tree ($O(k)$) :** where k is key length, not dataset size. Ideal if transactions are looked up by phone number prefix or partial ID.
- **Bloom Filter ($O(k)$ probabilistic):** a space-efficient structure that answers 'does this ID definitely not exist?' in constant time with zero false negatives useful as a pre-filter before hitting a database.

5. Conclusion

The MoMo SMS REST API demonstrates a complete CRUD lifecycle over 1,691 real mobile money records, secured with HTTP Basic Authentication and served via Python's built-in `http.server`. The DSA benchmark confirmed that dictionary-based lookup ($O(1)$) outperforms linear search ($O(n)$) by up to **739x** at this dataset size, a gap that would grow further at production scale.

From a security standpoint, Basic Auth is a pragmatic starting point for development but must be replaced before any production deployment. JWT with short-lived tokens and rotating refresh tokens is the recommended upgrade, providing credential-free requests, built-in expiry, and role-based access control with minimal integration complexity.

The combination of efficient in-memory indexing (dictionary lookup), RESTful resource design, and a clear path to stronger authentication positions this system for a clean migration to a persistent database and token-based auth as the next iteration.